Save Point

Spaced-Repetition Scheduling (FSRS)

AlotaNotes Quiz App · Prepared for Logan Young, Save Point Limited · 27 October 2026

Implemented in `lib/core/scheduling.dart`, a pure wrapper (no I/O, every timestamp passed in) around the `fsrs` package: the Open Spaced Repetition project's own Dart port of FSRS, the Free Spaced Repetition Scheduler that Anki uses. It replaced the fixed `1, 2, 3, 5, 7, 14, 30`-day ladder in schema v2.

What FSRS tracks

Every reviewable item is an `SrsCard` (`lib/core/models.dart`). Both quiz questions and tracked problems have one:

Field	Meaning
`state`	1 learning, 2 review, 3 relearning (after a lapse)
`step`	position in the short learning/relearning steps (null in review)
`stability`	days until recall probability falls to 90%
`difficulty`	1-10, how hard this item is for you
`due`	when it should come back
`lastReview`	when it was last reviewed

The forgetting curve: recall probability `t` days after the last review is `R = (1 + factor * t / S) ^ decay` (`retrievability()`, which uses fractional days so recall visibly drops within the first day). A review is scheduled for the moment `R` reaches the **target recall**, 90% by default, adjustable from 75% to 97% in Settings ("Memory"). A higher target means more frequent reviews.

Ratings

Every review is graded **Again / Hard / Good / Easy** (`ReviewRating`, values 1-4, the same as FSRS).

- **Quiz questions** are still auto-graded (`core/grading.dart`). A right answer *suggests* Good and a wrong one *suggests* Again (`suggestedRating()`). The suggestion is highlighted but the user taps the rating, so a lucky guess can be marked Hard and an instant recall Easy. Nothing is saved until a rating is tapped.
- **Problems** are always self-rated in the log-attempt sheet, with the suggestion pre-selected from solved / not solved.

Each rating button shows when that rating would bring the item back (`previewNextDue()`, formatted by `formatInterval()`: `10m`, `4d`, `2mo`). Fuzzing is **off**, so the preview is exactly what will be scheduled.

Lifecycle of a card

- **New** (`newCard()`): learning state, step 0, **due straight away**. This differs from the old ladder, where new questions waited a day.
- **Learning steps** of 1 and 10 minutes cover the first same-session repetitions. Good on a new card means 10 minutes; Easy skips learning (about 16 days).
- **Review**: intervals then grow with stability (days, weeks, months).
- **Again in review** is a lapse: relearning, back in 10 minutes, with lower stability.

A problem has no card at all (`card == null`) until its first attempt is logged, so an untried problem is never "due".

API

```
fsrs.Scheduler buildScheduler({double desiredRetention})
SrsCard newCard(DateTime now)
SrsCard reviewCard(SrsCard card, ReviewRating rating, DateTime reviewedAt, {scheduler})
Map<ReviewRating, DateTime> previewNextDue(SrsCard card, DateTime at, {scheduler})
double retrievability(SrsCard card, DateTime now, {scheduler})
ReviewRating suggestedRating(bool wasCorrect)
bool isDue(DateTime due, DateTime now)
String formatInterval(Duration d)
SrsCard cardFromLegacy({intervalIndex, nextDueAt, lastAskedAt})
```

Callers: `QuizScreen._rate()` (questions) and `ProblemDetailScreen._logAttempt()` (problems). Each computes the new card, saves it (`QuestionsRepo.updateScheduling()` / `ProblemsRepo.recordAttempt()`), and appends a history row with the rating and the resulting due date and stability.

Migrating from the old ladder

`openAppDatabase()` upgrades a v1 database in place (`migrateSchema()` in `db.dart`), keeping every row and id. Each question gets `cardFromLegacy()`: never answered becomes a new card; answered becomes a review card whose stability is the interval it had earned (for example, index 3 gives 5 days), at medium difficulty (5), **keeping its existing due date**. Old history rows get the rating auto-grading would have suggested (right gives Good, wrong gives Again).

Default parameters

The 21 FSRS weights are the package defaults. They aren't personalised: FSRS can optimise them from a user's own review history, which `question_history` and `problem_attempts` already record. See `future_work.md`.